Спецификация ОСРВ

Оглавление

1. Введение 3
1.1 Список сокращений 3
1.2 Назначение документа 3
1.3 Структура документа 3
2. Общие положения 4
3. Управление задачами 5
3.1 Состояния задачи 5
3.2 Активизация задачи 5
3.3 Приоритеты задач и переключение задач 6
3.4 Завершение задачи 6
4. Управление ресурсами 7
4.1 Организация управления ресурсами 7
4.2 Ограничения при управлении ресурсами 8
4.3 Протокол наивысшего приоритета 8
5. Спецификация API 9
5.1 Типы данных 9
5.2 Макрос TASK 10
5.3 DeclareTask 11
5.4 ActivateTask 11
5.5 TerminateTask 11
5.6 DeclareResource 12
5.7 GetResource 12
5.8 Release Resource 13
5.9 StartOS 13
5.10 ShutdownOS 13
6. Дополнительные требования к ОСРВ 14
6.1 Функциональные требования 14
6.2 Требования к языку программирования 14
6.3 Требования к файловой структуре проекта 14

1. Введение

1.1 Список сокращений

Сокращение	Определение
ОС	операционная система
ОСРВ	операционная система реального времени
HLP	протокол наивысшего приоритета (highest locker protocol)
API	интерфейс для программирования приложений (application programming interface)
ISO	Международная Организация по Стандартам (International Standards Organization)
ANSI	Американский Национальный Институт Стандартов (American National Standards Institute)

1.2 Назначение документа

В данном документе описываются требования, которым в обязательном порядке (если не оговорено обратное) должны удовлетворять все проекты ОСРВ, выполненные студентами в рамках учебного курса «Архитектура программных систем». Кроме того, в нём приводятся некоторые рекомендации по разработке проекта, целью которых — помочь студенту принять ключевые решения, связанные с архитектурой разрабатываемой ОСРВ. Выполнение этих рекомендаций не является обязательным требованием к учебному проекту.

1.3 Структура документа

В этом разделе содержится краткое описание последующих глав данного документа.

- **Глава 2, Общие положения.** Здесь описаны основные характеристики и подсистемы ОСРВ.
- **Глава 3, Управление задачами.** В этой главе рассматривается жизненный цикл задачи, приоритеты и вопросы планирования.
- **Глава 4, Управление ресурсами.** В этой главе рассматривается организация критических секций и протокол HLP, применяемый для управлении ресурсами.
- **Глава 5, Спецификация API.** В этой главе приведена подробная спецификация системных сервисов ОС.
- **Глава 6, Дополнительные требования к ОСРВ.** Здесь сформулированы дополнительные требования к функциональности ОС, языку программирования и файловой структуре проекта.

2. Общие положения

ОСРВ включает в себя управляющие модули и набор системных сервисов, доступных пользовательским приложениям.

Все объекты ОСРВ создаются статически на этапе компиляции и редактирования связей.

Все приложения, написанные для ОСРВ, удовлетворяющей данной спецификации, должны без ограничений переноситься на любую другую ОСРВ, также удовлетворяющую данной спецификации. Это достигается посредством строгого определения всех системных сервисов, их поведения и ограничений на их использование.

ОСРВ включает в себя подсистемы, подробно рассматриваемые в последующих главах, обеспечивающие следующие возможности:

- Управление задачами
- Синхронизацию задач посредством использования разделяемых ресурсов

3. Управление задачами

3.1 Состояния задачи

В любой момент времени задача должна находиться в одном (и только одном) из следующих состояний:

- Выполняется (running). Задача назначена на выполнение и использует процессор для последовательного исполнения своих инструкций. Только одна задача может находиться в этом состоянии в заданный момент времени.
- Активна (ready). Задача получила все необходимые для выполнения ресурсы (память, стек и т. п.) и ожидает назначения на выполнение. Только из этого состояния задача может перейти в состояние running.
- Приостановлена (suspended, применяется также термин inactive). В этом состоянии задача пассивна, не занимает системных ресурсов и не может быть назначена на выполнение.

3.2 Активизация задачи

Активизация задачи осуществляется сервисом ActivateTask. При этом задача переходит из состояния suspended в состояние ready, после чего может быть назначена на выполнение. *Повторная активизация задачи пользовательским приложением не допускается, поведение ОСРВ в этом случае не определено.*

Передавать задаче при активизации определённый набор параметров (как функции C) не допускается; поэтому для этой цели следует использовать другие механизмы (например, глобальные переменные).

3.3 Приоритеты задач и переключение задач

Поскольку ОСРВ выполняется в однопроцессорной системе, но может выполнять более одной задачи, необходимо назначать задачи на процессор по очереди. Последовательность назначения задач на процессор определяется *планировщиком* на основании приоритетов активных задач. Собственно переключение с одной задачи на другую выполняется *диспетчером*.

Приоритеты задач назначаются статически и не могут изменяться пользователем в процессе выполнения. Тем не менее, ОС в некоторых случаях может динамически изменять приоритеты задач (см. раздел 4.3). Большее число соответствует более высокому приоритету задачи. Нулевой приоритет является самым низким.

Планировщик и диспетчер должны реализовывать *вытесняемое планирование задач*. Это означает, что задача, выполняющаяся на процессоре, может быть приостановлена (переведена в состояние ready) в любой момент, если в систему поступает более высокоприоритетная задача.

Если две задачи имеют одинаковый приоритет, то в первую очередь должна выполниться задача, которая была активизирована раньше.

Диспетчер вызывается в одном из следующих случаев:

- Завершение задачи
- Активизация задачи другой задачей
- Освобождение ресурса одной из задач.

3.4 Завершение задачи

В рассматриваемой ОСРВ задача не может завершить другую задачу. Кроме того, *каждая задача обязана завершить саму себя вызовом системного сервиса TerminateTask*. В противном случае поведение ОС не определено.

4. Управление ресурсами

4.1 Организация управления ресурсами

Управление ресурсами используются для организации совместного использования разделяемых ресурсов (общих областей памяти, аппаратных средств и т. п.) несколькими задачами.

Для управления ресурсами используются системные сервисы GetResource и ReleaseResource. Вызовы этих сервисов ограничивают *критическую секцию* в пользовательской задаче. *Любое использование ресурса задачей допускается только внутри критической секции*.

ОСРВ должна обеспечивать неограниченную вложенность критических секций. Другими словами, задача может захватить и удерживать произвольное количество ресурсов одновременно. На порядок освобождения захваченных ресурсов также не накладывается никаких ограничений. Это означает, что оба фрагмента программы, показанных в таблице 2, являются корректными.

Таблица 1

… GetResource(res1); … GetResource(res2); … ReleaseResource(res2); … ReleaseResource(res1); …	… GetResource(res1); … GetResource(res2); … ReleaseResource(res1); … ReleaseResource(res2); …

4.2 Ограничения при управлении ресурсами

Хотя вложенный захват ресурсов является допустимым, запрещается вложенный захват задачей одного и того же ресурса.

Не допускается применение сервиса TerminateTask внутри критической секции. Перед завершением задача обязана освободить все захваченные ресурсы.

4.3 Протокол наивысшего приоритета

Реализация протокола наивысшего приоритета (Highest Locker Protocol, HLP) является обязательным требованием к ОСРВ.

Его применение гарантирует, что в процессе работы:

- две задачи не могут одновременно обратиться к одному и тому же ресурсу
- не возникает неограниченной инверсии приоритетов
- не возникает тупиков в ходе выполнения программы

HLP предписывает следующий подход при управлении ресурсами.

- На этапе построения системы (но не на этапе выполнения) каждому ресурсу назначается наивысший приоритет (ceiling priority, CP). Наивысший приоритет вычисляется по формуле $CP_k = \max_i P_{ik}$, где CP_k - наивысший приоритет k-го ресурса, а P_{ik} - приоритет i-й задачи, использующей k-й ресурс.
- Если приоритет задачи, захватывающей ресурс, меньше наивысшего приоритета данного ресурса, то её приоритет увеличивается до значения наивысшего приоритета этого ресурса.
- После освобождения ресурса приоритет задачи принимает значение, которое он имел до захвата этого ресурса.

Более подробная информация о HLP содержится в курсе лекций по ОСРВ.

5. Спецификация API

В этой главе приводится описание API, предоставляемого ОСРВ пользовательским приложениям. **Все системные сервисы, описанные ниже, должны быть реализованы в полном соответствии со спецификацией.** Сервисы оформляются как функции языка С или как макросы препроцессора.

Каждое описание состоит из следующих разделов:

Синтаксис	Синтаксис вызова в стиле языка С
Параметры	Входные параметры, передаваемые сервису. Выходные параметры не используются, и все сервисы обязаны возвращать void
Описание	Описание функций, которые должен выполнять сервис ОС
Особенности	Особенности применения сервиса ОС и ограничения при его использовании

5.1 Типы данных

При именовании типов используются следующая договорённость: имя типа должно начинаться с префикса “T”. Типы, применяемые в ОСРВ, перечислены в таблице 3.

Таблица 2

Тип	Описание
TTask	Идентификатор задачи
TResource	Идентификатор ресурса

5.2 Макрос TASK

Макрос TASK используется для описания задачи (примерно так же, как описываются функции в языке C). Он имеет следующий синтаксис:

```
TASK(TaskID, prior)
{
<тело задачи>

TerminateTask();
}
```

Здесь TaskID — любой корректный идентификатор языка C, а prior — приоритет задачи. Задача с идентификатором TaskID должна быть предварительно объявлена с использованием сервиса DeclareTask (см. раздел 6.3). Удобство такого подхода заключается в том, что в дальнейшем задачу можно активизировать с использованием того же идентификатора TaskID, например:

```
DeclareTask(task1);
DeclareTask(task2);

TASK(task1)
{
ActivateTask(task2);
TerminateTask();
}
```

5.3 DeclareTask

Синтаксис	DeclareTask(TaskID)
Параметры	TaskID — идентификатор задачи, удовлетворяющий стандартным требованиям языка C.
Описание	Осуществляет регистрацию задачи в системе (назначение подобно объявлению глобальных функций в языке C). Должен вызываться ДО использования задачи в коде пользовательского приложения.
Особенности	-

5.4 ActivateTask

Синтаксис	void ActivateTask(TTask task)
Параметры	task — идентификатор задачи, подлежащей активизации
Описание	Переводит задачу из состояния suspended в состояние ready
Особенности	Повторная активизация задачи не допускается.

5.5 TerminateTask

Синтаксис	void TerminateTask(void)
Параметры	-
Описание	Завершает выполнение текущей задачи, переводя её из состояния running в состояние suspended. При этом освобождаются все системные ресурсы, занятые задачей.
Особенности	Вызов этого сервиса всегда является последней инструкцией, которую выполняет задача. Запрещается вызывать TerminateTask из критической

секции задачи.

Завершение задачи без обращения к этому сервису также категорически запрещается.

5.6 DeclareResource

Синтаксис	DeclareResource(ResourceID, int CP)
Параметры	ResourceID — идентификатор ресурса, удовлетворяющий стандартным требованиям языка C. CP — наивысший приоритет ресурса.
Описание	Осуществляет регистрацию ресурса в системе (назначение подобно объявлению глобальных переменных в языке C). Должен вызываться ДО использования ресурса в коде пользовательского приложения.
Особенности	-

5.7 GetResource

Синтаксис	void GetResource(TResource res)
Параметры	res — идентификатор ресурса, запрашиваемого задачей.
Описание	Вызов этого сервиса отмечает вход в критическую секцию. Выход из этой критической секции всегда должен осуществляться вызовом ReleaseResource (см. раздел 6.8).
Особенности	Сервис реализуется с использованием протокола наивысшего приоритета (см. раздел 5.3). Вызов TerminateTask из критической секции не допускается. Запрещён также вложенный захват одного и того же

ресурса.

5.8 Release Resource

Синтаксис	void ReleaseResource(TResource res)
Параметры	res — идентификатор ресурса, подлежащего освобождению.
Описание	Вызов этого сервиса отмечает выход из критической секции и должен сопровождаться предварительным вызовом GetResource.
Особенности	Запрещено освобождать ресурс, который ранее не был захвачен.

5.9 StartOS

Синтаксис	void StartOS(TTask task)
Параметры	task – задача, которую необходимо назначить на выполнение после инициализации системы.
Описание	Осуществляет необходимую инициализацию и запускает ОС на выполнение.
Особенности	Вызов этого сервиса возможен только «извне» ОС, то есть когда она ещё не начала свою работу. Возврат из этого сервиса осуществляется только после вызова ShutdownOS из задачи или внутренней функции ОС.

5.10 ShutdownOS

Синтаксис	void ShutdownOS(void)
Параметры	-
Описание	Используется для немедленного завершения работы системы.
Особен-ти	-

6. Дополнительные требования к ОСРВ

В этой главе приводятся дополнительные требования к ОСРВ, которые не были названы в предыдущих разделах.

6.1 Функциональные требования

Максимальное количество задач	32
Максимальное количество ресурсов	16
Максимальное количество *активных* задач	16
Число приоритетов	16
Максимальное количество захваченных ресурсов	8

6.2 Требования к языку программирования

Проект должен быть реализован на языке C. Использование языка C++ не допускается. Весьма желательно, чтобы исходный код удовлетворял стандарту ISO/ANSI C. Желательно также исключить использование ассемблера (как внешнего, так и встроенного) с целью повышения переносимости продукта с одной платформы на другую.

6.3 Требования к файловой структуре проекта

К файловой структуре проекта предъявляются следующие требования. Модули операционной системы должны целиком размещаться в пяти файлах, написанных на языке C: **global.c**, **os.c**, **resource.c** и **task.c**. Исключение из проекта хотя бы одного из этих файлов не допускается. Не разрешается также вводить в проект дополнительные файлы.

Рекомендуется следующее распределение модулей ОС по файлам проекта:

Файл	Что содержит
global.c	Описание всех глобальных переменных проекта
os.c	Модуль, предназначенный для управления ОС
resource.c	Модуль, предназначенный для управления ресурсами
task.c	Модуль, предназначенный для управления задачами

Тем не менее, соблюдение этих рекомендаций не является обязательным. Можно, например, поместить весь полезный код операционной системы в один файл **os.c**, а остальные оставить пустыми.

Все макросы, описания типов, декларации функций и другие объекты языка С, которые необходимо включать в пользовательское приложение, должны быть сосредоточены в одном заголовочном файле **rtos_api.h**.

Для организации взаимодействия модулей операционной системы между собой можно использовать любое количество заголовочных файлов. Их названия и содержимое не регламентируется.